

The “Issue Tracker” MVP

Objective

Build a minimal but production-minded Issue Tracker that supports users, projects, and issues, with real-time updates and role-based access.

1. Write this app using Angular 20 & Java (JDK 17).
 2. You can use AI.
 3. Questions? State any assumptions you make and move forward.
 4. We're looking for a quality over speed of solution.
 5. Return your deliverable within 6 hours.
-

Scope (MVP):

- **Auth:** Sign up / login (email+password) or OAuth; password hashing; basic session/JWT.
- **Domain:**
 - `Project { id, name, ownerId }`
 - `Issue { id, projectId, title, description, status: [Open|InProgress|Closed], priority, assigneeId, tags[], createdAt, updatedAt }`
- **API:** CRUD for Projects and Issues; server-side pagination & filtering (`status`, `priority`, `assigneeId`, `tag`, text search on `title`).
- **UI:**
 - Project list & detail.

- Issue list with filters, sort, and text search.
- Issue detail view with edit, comment thread, and activity log.
- **Real-time:** Push updates when an issue changes (WebSocket/SSE).
- **Data model:** Relational or document—justify the choice and indexing strategy.
- **Quality gates:**
 - Unit tests for core business logic.
 - One integration test hitting API + DB.
 - Basic input validation and error handling.
- **Ops hygiene:**
 - `Dockerfile` + `docker-compose` (or equivalent) to run API, DB, and web app.
 - Seed script for demo data.
 - Minimal README: stack, run instructions, trade-offs, and “If I had 2 more days” section.

Constraints (to surface judgment):

- Implement either **RBAC** (Owner, Maintainer, Reporter) or **optimistic concurrency** on issue edits.
- Show **one performance consideration** (e.g., N+1 query avoided, useful index, cache/memoization).
- Add **structured logging** and a **health check** endpoint.

Evaluation signals:

- Sensible domain boundaries, clear layering, testable code, defensive inputs, clean migrations/seed data.
- UI that’s not pretty-for-pretty’s sake but functionally crisp: empty/loading/error states, optimistic UI where appropriate.
- Thoughtful README with explicit trade-offs.